

Aula 1 - Jogo Snake com personalizacao visual

Guia do Aluno - versao detalhada

O que voce vai construir

Um jogo Snake que roda no seu navegador: a cobra anda, come, cresce e morre - e com a aparencia que VOCE escolher. Voce abre o index.html, digita seu nome, clica em Jogar e controla a cobra com as setas ou as teclas W, A, S, D.

O jogo ja abre e mostra a cobra parada. Seu trabalho e implementar 3 funcoes dentro do arquivo game.js (marcadas com // TODO) que fazem a cobra ganhar vida. No final, voce personaliza tudo no config.js. Os outros arquivos ja estao prontos.

Antes de comecar: como o jogo funciona por dentro

Leia esta secao com calma. Ela explica o 'mundo' do jogo. Depois dela, os 3 desafios viram quase obvios.

1) A grade e o sistema de coordenadas

O tabuleiro e uma GRADE de quadradinhos (celulas). Tem COLUNAS de largura e LINHAS de altura (por padrao 30 x 30).

Cada celula tem uma posicao (x, y). x e a coluna (esquerda->direita), y e a linha (cima->baixo).

ATENCAO: o canto de cima-a-esquerda e (0,0). O x cresce para a DIREITA e o y cresce para BAIXO. Por isso 'ir para cima' e DIMINUIR o y (y - 1).

2) Como a cobra e guardada na memoria

A cobra e uma LISTA (array) de pontos. Cada ponto e um objeto { x: ..., y: ... }.

O primeiro item da lista, cobra[0], e sempre a CABECA. O resto e o corpo.

Exemplo: cobra = [{x:5,y:5}, {x:4,y:5}, {x:3,y:5}] e uma cobra de 3 pedacos, com a cabeca em (5,5), deitada na horizontal.

3) As variaveis de estado (a 'situacao' do jogo)

cobra - a lista de pedacos (cobra[0] = cabeca).

direcao - para onde a cobra vai agora: 'cima','baixo','esquerda','direita'.

proximaDirecao - a direcao que o teclado escolheu, aplicada no proximo passo.

comida - a posicao da comida: { x, y }.

comidaEspecial - true se a comida atual vale 3x.

score, nivel - pontuacao e nivel de velocidade.

jogoAtivo - true enquanto o jogo roda.

COLUNAS, LINHAS - tamanho da grade (vem do CONFIG).

4) O loop do jogo (o 'tick')

A cada VELOCIDADE milissegundos, a funcao tick() roda automaticamente. Ela e o coracao do jogo e faz, nesta ordem:

1. aplica a direcao escolhida no teclado;
2. chama moverCobra() <- SEU Desafio 1;
3. chama verificarColisao() <- SEU Desafio 2;
4. ve se a cabeca caiu na comida (e cresce);
5. desenha tudo na tela.

Voce nao escreve o tick(); ele ja esta pronto e CHAMA as suas funcoes.

Funcoes e ferramentas que ja existem e voce vai usar

- cobra.unshift(p) - adiciona o ponto p no INICIO da lista (vira a nova cabeca).
- cobra.pop() - remove o ULTIMO ponto da lista (a cauda).
- comportamentoDeBorda(cabeca) - decide o que fazer quando a cobra passa da parede: devolve null (morreu) ou uma posicao nova (modo atravessar parede).
- encerrarJogo() - termina a partida e mostra a tela de game over.
- CONFIG - o objeto com a sua personalizacao (CONFIG.wraparound, CONFIG.chance_comida_especial...).
- Math.random() - sorteia um numero entre 0 e 1. Math.floor(n) - arredonda para baixo.

5) Direcao -> movimento (dx, dy)

Para mover, transformamos a direcao num deslocamento (dx, dy) que sera somado a cabeca:

'cima' -> dx = 0, dy = -1 (y diminui = sobe)

'baixo' -> dx = 0, dy = +1

'esquerda' -> dx = -1, dy = 0

'direita' -> dx = +1, dy = 0

Os 3 desafios (passo a passo)

Desafio 1: moverCobra() - fazer a cobra andar

Entendendo o problema

A cada tick, a cobra precisa dar um passo. Mover uma lista de 3, 10 ou 50 pedacos parece trabalhoso - mas existe um truque elegante.

Em vez de mover cada pedaco, a gente ADICIONA uma cabeca nova na direcao do movimento e REMOVE o ultimo pedaco. Visualmente, a cobra desliza. E como uma fila de pessoas: entra uma na frente, sai uma atras, e a fila 'anda'.

A funcao moverCobra() nao devolve nada; ela MODIFICA o array global 'cobra'.

O que voce precisa saber antes

A regra de ouro: unshift + pop

cobra.unshift(novaCabeca) coloca a nova cabeca no inicio.

cobra.pop() remove o ultimo pedaco (a cauda).

Os dois juntos = a cobra anda uma casa, mantendo o mesmo tamanho.

Quando a cobra COME, outra parte do codigo devolve um pedaco - por isso ela cresce. Aqui voce sempre remove a cauda normalmente.

A variável 'direcao' já diz para onde ir. COLUNAS e LINHAS dizem o tamanho da grade. comportamentoDeBorda() já existe e cuida da parede.

Construindo passo a passo

Passo 1 — Descubra dx e dy a partir da direcao

Comece os dois em 0. So uma das condicoes vai mudar um deles. Lembre que 'cima' e dy = -1 (porque o y cresce para baixo).

```
var dx = 0, dy = 0;
if (direcao === "cima")    dy = -1;
if (direcao === "baixo")   dy = 1;
if (direcao === "esquerda") dx = -1;
if (direcao === "direita") dx = 1;
```

Passo 2 — Calcule a posicao da nova cabeca

cobra[0] e a cabeca atual. A nova cabeca e a cabeca atual somada ao deslocamento.

```
var cabeca = cobra[0];
var nova = { x: cabeca.x + dx, y: cabeca.y + dy };
```

Passo 3 — Trate a borda (parede)

A nova cabeca pode ter saído da grade. 'Estar fora' significa x menor que 0, ou x maior/igual a COLUNAS, ou o mesmo para y e LINHAS.

Se saiu, pergunte a comportamentoDeBorda(nova): se devolver null, a cobra morreu (chame encerrarJogo() e saia com return). Se devolver uma posicao, use essa posicao (e o modo 'atravessar parede').

```
var fora = (nova.x < 0 || nova.x >= COLUNAS ||
            nova.y < 0 || nova.y >= LINHAS);
if (fora) {
  var r = comportamentoDeBorda(nova);
  if (r === null) { encerrarJogo(); return; }
  nova = r;
}
```

Passo 4 — Ande: cabeca nova na frente, remove a cauda

unshift põe a nova cabeca no início; pop tira o último pedaço.

```
cobra.unshift(nova);
cobra.pop();
```

Confira: a função completa

```
function moverCobra() {
  var dx = 0, dy = 0;
  if (direcao === "cima")    dy = -1;
  if (direcao === "baixo")   dy = 1;
  if (direcao === "esquerda") dx = -1;
  if (direcao === "direita") dx = 1;

  var cabeca = cobra[0];
  var nova = { x: cabeca.x + dx, y: cabeca.y + dy };

  var fora = (nova.x < 0 || nova.x >= COLUNAS ||
             nova.y < 0 || nova.y >= LINHAS);
  if (fora) {
    var r = comportamentoDeBorda(nova);
    if (r === null) { encerrarJogo(); return; }
    nova = r;
  }

  cobra.unshift(nova);
  cobra.pop();
}
```

Como saber que funcionou

A cobra desliza pela tela ao clicar em Jogar. Ao bater na parede, o jogo encerra. (A morte por morder o proprio corpo vem no Desafio 2.)

Se der erro

Mensagem/sintoma: "Cannot read properties of undefined (reading 'x')"

Causa: voce usou a variavel 'nova' antes de cria-la.

Solucao: crie 'var nova = { x: cabeca.x + dx, y: cabeca.y + dy };' ANTES de usar 'nova'.

Se der erro

Mensagem/sintoma: a cobra cresce sem parar (vira uma cobra gigante).

Causa: voce esqueceu o cobra.pop() - sem ele a cauda nunca e removida.

Solucao: garanta que ha um cobra.pop() depois do cobra.unshift(nova).

Se der erro

Mensagem/sintoma: a cobra anda 'de re' ou nao se mexe.

Causa: voce usou push (adiciona no fim) em vez de unshift (adiciona no inicio).

Solucao: use cobra.unshift(nova) para por a cabeca na frente.

Desafio 2: verificarColisao() - saber quando a cobra bate

Entendendo o problema

A cobra morre de dois jeitos: batendo na parede ou mordendo o proprio corpo. Esta funcao precisa RESPONDER uma pergunta: a cobra bateu? Ela devolve true (bateu) ou false (livre).

Quem chama essa funcao e o tick(): se voltar true (e a cobra nao estiver com escudo), ele chama encerrarJogo(). Voce so precisa devolver a resposta certa.

O que voce precisa saber antes

Por que o corpo começa em i = 1?

cobra[0] é a própria cabeça. Se comparássemos a cabeça com cobra[0], elas seriam sempre iguais - um 'falso positivo' de colisão.

Por isso percorremos o corpo a partir de i = 1 (o primeiro pedaço DEPOIS da cabeça).

Comparar pontos: x E y

Dois pontos são a MESMA célula só quando x e y batem ao mesmo tempo.

Use &&: (cobra[i].x === cabeça.x && cobra[i].y === cabeça.y).

Use === (três iguais) para comparar valores em JavaScript.

Se CONFIG.wraparound for true, a cobra atravessa a parede e não morre nela (o moverCobra já teletransportou). Por isso só checamos a parede quando wraparound é false.

Construindo passo a passo**Passo 1 — Pegue a cabeça**

É o ponto que vamos comparar com tudo.

```
var cabeça = cobra[0];
```

Passo 2 — Cheque a parede (apenas se não for wraparound)

Se a cabeça saiu da grade, bateu na parede: devolva true na hora.

```
if (!CONFIG.wraparound) {
  if (cabeça.x < 0 || cabeça.x >= COLUNAS ||
      cabeça.y < 0 || cabeça.y >= LINHAS) {
    return true;
  }
}
```

Passo 3 — Cheque o próprio corpo

Percorra do pedaço 1 até o fim. Se algum pedaço está na mesma célula da cabeça, a cobra se mordeu: devolva true.

```
for (var i = 1; i < cobra.length; i++) {
  if (cobra[i].x === cabeça.x && cobra[i].y === cabeça.y) {
    return true;
  }
}
```

Passo 4 — Se nada bateu, está livre

Chegou até aqui sem return? Então não houve colisão.

```
return false;
```

Confira: a função completa

```
function verificarColisao() {
  var cabeca = cobra[0];

  if (!CONFIG.wraparound) {
    if (cabeca.x < 0 || cabeca.x >= COLUNAS ||
        cabeca.y < 0 || cabeca.y >= LINHAS) {
      return true;
    }
  }

  for (var i = 1; i < cobra.length; i++) {
    if (cobra[i].x === cabeca.x && cobra[i].y === cabeca.y) {
      return true;
    }
  }
  return false;
}
```

Como saber que funcionou

Ao morder o proprio corpo (ou bater na parede), aparece a tela de game over.

Se der erro

Mensagem/sintoma: o jogo vai direto para game over assim que comeca.

Causa: o loop comecou em i = 0, comparando a cabeca com ela mesma.

Solucao: comece o loop em i = 1: 'for (var i = 1; ...'.

Se der erro

Mensagem/sintoma: a cobra nunca morre ao se morder.

Causa: voce esqueceu o 'return true;' dentro do if, ou comparou so x (ou so y).

Solucao: compare x E y com && e devolva true quando baterem.

Desafio 3: gerarComida() - sortear onde a comida aparece

Entendendo o problema

A comida deve nascer numa celula VAZIA - nunca em cima da cobra. Como achar uma celula livre? Sorteamos uma posicao; se cair na cobra, sorteamos de novo, ate achar uma livre.

No fim, esta funcao define duas variaveis globais: 'comida' (um ponto {x,y}) e 'comidaEspecial' (true se valer 3x).

O que voce precisa saber antes

Sortear uma celula valida

Math.random() da um numero entre 0 e 1 (ex: 0.732).

Multiplicando por COLUNAS e aplicando Math.floor, vira um inteiro de 0 ate COLUNAS-1. Ex:

Math.floor(0.732 * 30) = 21.

Sem Math.floor, a comida cairia 'entre' celulas (numero quebrado).

do...while: tente, depois verifique

do { ... } while (condicao) executa o bloco UMA vez e repete ENQUANTO a condicao for verdadeira.

Perfeito para 'sorteie; se caiu na cobra, sorteie de novo'. A condicao do while deve ser 'ainda esta em cima da cobra?'.

Construindo passo a passo

Passo 1 — Declare as variaveis de apoio

'pos' guarda a posicao sorteada; 'emCimaDaCobra' diz se ela bateu na cobra.

```
var pos, emCimaDaCobra;
```

Passo 2 — Sorteie uma posicao (dentro do do...while)

Use Math.floor + Math.random para x e y.

```
pos = {  
  x: Math.floor(Math.random() * COLUNAS),  
  y: Math.floor(Math.random() * LINHAS)  
};
```

Passo 3 — Verifique se a posicao caiu na cobra

Comece supondo que nao caiu. Percorra a cobra; se algum pedaco esta na posicao sorteada, marque emCimaDaCobra = true e pare (break).

```
emCimaDaCobra = false;  
for (var i = 0; i < cobra.length; i++) {  
  if (cobra[i].x === pos.x && cobra[i].y === pos.y) {  
    emCimaDaCobra = true;  
    break;  
  }  
}
```

Passo 4 — Junte tudo num do...while

Os passos 2 e 3 ficam DENTRO do 'do'. A condicao do 'while' repete enquanto a posicao estiver em cima da cobra.

```
do {  
  // passo 2 (sortear) e passo 3 (verificar) aqui dentro  
} while (emCimaDaCobra);
```

Passo 5 — Guarde a comida e sorteie se e especial

A posicao final (livre) vira a comida. Depois, com a probabilidade do CONFIG, decida se ela e especial (vale 3x). Math.random() < 0.15 da 'true' em ~15% das vezes.

```
comida = pos;  
comidaEspecial = Math.random() < CONFIG.chance_comida_especial;
```

Confira: a funcao completa

```
function gerarComida() {
  var pos, emCimaDaCobra;
  do {
    pos = {
      x: Math.floor(Math.random() * COLUNAS),
      y: Math.floor(Math.random() * LINHAS)
    };
    emCimaDaCobra = false;
    for (var i = 0; i < cobra.length; i++) {
      if (cobra[i].x === pos.x && cobra[i].y === pos.y) {
        emCimaDaCobra = true;
        break;
      }
    }
  } while (emCimaDaCobra);

  comida = pos;
  comidaEspecial = Math.random() < CONFIG.chance_comida_especial;
}
```

Como saber que funcionou

A comida aparece em lugares diferentes a cada vez; comer faz a cobra crescer e o score subir. As vezes a comida aparece de outra cor (a especial, que vale 3x).

Se der erro

Mensagem/sintoma: a comida nunca aparece ou o navegador trava (congela).

Causa: a condicao do while esta invertida (repete enquanto esta LIVRE = laço infinito).

Solucao: a condicao deve repetir ENQUANTO 'emCimaDaCobra' for true.

Se der erro

Mensagem/sintoma: a comida aparece 'entre' os quadradinhos ou em posicoes erradas.

Causa: voce esqueceu o Math.floor no sorteio.

Solucao: envolva o sorteio com Math.floor(Math.random() * COLUNAS).

Personalize seu jogo (deixe com a SUA cara)

Abra o config.js - esse arquivo e todo seu, e mexer nele NUNCA quebra a logica do jogo. Mude pelo menos CINCO coisas, sendo pelo menos DUAS de layout. Salve e aperte F5 no navegador.

Visual basico

- cor_cobra_cabeca e cor_cobra_corpo - cores em #hexadecimal (ex: #4ecca3).
- velocidade_inicial - ms entre cada passo. MENOR = MAIS RAPIDO (minimo 60).
- emoji_morte e mensagem_game_over - o que aparece quando a cobra morre.

Layout e aparencia (recomendado)

- titulo_jogo - troque pelo nome do SEU jogo.
- fonte_jogo / fonte_score - ex: "Courier New, monospace" para um visual retro.
- cor_texto, cor_texto_score, cor_painel, cor_botao - o tema de cores da interface.
- cor_borda_canvas, largura_borda_canvas, raio_borda_canvas - a moldura do tabuleiro.
- posicao_painel - "esquerda" ou "direita" para mover o placar de lado.

Bonus de mecanica

- wraparound: true - a cobra atravessa a parede e sai do outro lado.

- chance_comida_especial - proporcao de comidas que valem 3x (0 a 1).

Como saber que funcionou

Ao salvar e recarregar, TUDO muda na hora (titulo, fonte, cores, moldura, lado do placar) - sem tocar em nenhum outro arquivo.

Dica: monte um TEMA coerente (3 cores que combinem + 1 fonte). Essa identidade vai te acompanhar ate o torneio da Aula 4.

Para conferir se voce entendeu

1) Por que conseguimos mover a cobra so mexendo nas pontas da lista?

Resposta: porque adicionar uma cabeca na frente (unshift) e remover a cauda (pop) faz a lista inteira 'deslizar' uma casa.

2) Por que 'ir para cima' diminui o y?

Resposta: porque o y cresce para baixo (o topo da tela e $y = 0$).

3) Por que o loop de colisao comeca em $i = 1$ e nao em $i = 0$?

Resposta: $i = 0$ e a propria cabeca; compara-la com ela mesma daria colisao falsa.

4) Para que serve o do...while no gerarComida?

Resposta: para sortear de novo enquanto a posicao cair em cima da cobra, ate achar uma livre.

Glossario

Array (lista): uma sequencia ordenada de itens. A cobra e um array de pontos.

Objeto { x, y }: um pacotinho de dados com campos nomeados. Aqui guarda uma posicao.

Indice: a posicao de um item na lista, comecando em 0. cobra[0] e o primeiro.

Canvas: um quadro de desenho na pagina; o JavaScript pinta os quadradinhos nele.

tick / setInterval: um 'pulso' que dispara a cada X ms; a cada pulso o jogo avanca um passo.

unshift / pop / push: unshift poe no inicio; pop tira do fim; push poe no fim.

do...while: laco que executa uma vez e repete enquanto a condicao for verdadeira.

=== (tres iguais): compara valores em JavaScript de forma estrita (sem converter tipos).

&& (E logico): verdadeiro so quando as duas condicoes sao verdadeiras.

return: encerra a funcao e (as vezes) devolve um valor para quem a chamou.

Variavel CSS (--cor-fundo): um apelido de estilo; o ui.js copia os valores do config.js para eles.

Hexadecimal de cor (#4ecca3): a 'receita' da cor: quanto de vermelho, verde e azul misturar.